Best practice pour repository public

Workflow Git d'équipe : branches de travail, Pull Request avec 2 reviewers, merge classique.

Commentaire : que fait cette étape ?	Commandes du workflow
1. Cloner le dépôt Récupère une copie locale du dépôt public et se place dedans. À faire une seule fois par personne.	`git clone <url>` `cd <dossier-du-depot>`
2. Créer sa branche de travail Crée une nouvelle branche (une branche = un sujet) et bascule dessus. Ne jamais travailler directement sur main.	`git switch -c feature/nom-du-groupe`
3. Vérifier son travail Affiche les fichiers modifiés et le détail des changements. Contrôler qu'il ne reste que ce qui est voulu (pas de fichiers temporaires ni de données sensibles).	`git status` `git diff`
4. Commiter proprement Prépare les fichiers choisis (zone de staging) puis les enregistre dans l'historique local avec un message clair.	`git add <fichiers>` `git commit -m "Message clair"`
5. Mettre à jour sa branche avec main Télécharge les nouveautés du dépôt distant et les intègre dans sa branche. Les conflits éventuels se résolvent sur sa propre branche, jamais sur main.	`git fetch origin` `git merge origin/main`
6. Tester Lancer le projet ou les tests après la mise à jour, avant de partager son travail.	`(lancer les tests du projet)`
7. Pousser sa branche (jamais main) Publie la branche sur GitHub. L'option -u n'est nécessaire qu'au premier push ; ensuite git push suffit.	`git push -u origin feature/nom-du-groupe`
8. Ouvrir la Pull Request Sur GitHub : base main, comparaison = sa branche. Titre clair, courte description (quoi, pourquoi, comment tester) et désignation des 2 reviewers.	`(sur GitHub : New pull request)`

9. Traiter les retours de la revue
Corriger sur la même branche : la PR se met à jour automatiquement. Pas de force-push sans l'accord de l'équipe.
	`git add <fichiers>`
	`git commit -m "Corrige selon la revue"`
	`git push`

10. Fusionner après 2 approbations Merge depuis le bouton GitHub (merge classique, sans squash), une fois les 2 approbations obtenues et les contrôles au vert.	`(sur GitHub : Merge pull request)`

11. Nettoyer après le merge
Revient sur main, le met à jour (échoue s'il y a des commits locaux inattendus), supprime la branche locale fusionnée (-d est la version sécurisée) et retire les références aux branches distantes supprimées.
	`git switch main`
	`git pull --ff-only`
	`git branch -d feature/nom-du-groupe`
	`git fetch --prune`

Commentaire : que fait cette étape ?	Commandes du workflow

12. Pour le reste de l'équipe
Après chaque merge dans main, mettre à jour son main local puis l'intégrer dans sa propre branche (au plus tard avant d'ouvrir sa PR).
	`git fetch origin`
	`git switch main`
	`git pull --ff-only`
	`git switch feature/ma-branche`
	`git merge main`

Règles d'or

- Personne ne pousse directement sur main (activer la protection de branche : PR obligatoire, 2 approbations).
- Une branche = un sujet ; commits petits et messages clairs.
- On met à jour sa branche depuis main, et on intègre dans main uniquement via une PR.
- On ne réécrit pas l'historique d'une branche partagée.
- Les branches de travail sont supprimées après le merge ; les tags servent à marquer les versions importantes.
- En cas de doute : git status et git log --oneline --graph.